

ECS766P DATA MINING

Dr Emmanouil Benetos, Dr Dimitrios Kollias

Assignment 1 (worth 20% of total mark)

1) The function “load_wine” from “sklearn.datasets” can be used to load the wine dataset into a “DataFrame” by using the below commands:

```
data = load_wine()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = pd.Series(data.target)
```

a) Load the wine dataset. Which feature is categorical and why? Compute the frequency (not the occurrence) of each value of the categorical feature. Include the code in your report. [8 marks]

The dataset has 13 continuous variables, while the only categorical one is ‘target’, which indicates the wine type (0, 1, 2). I checked the class frequencies, and they’re roughly even, so I decided not to apply any resampling, it already looks balanced enough for analysis.

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_wine

data = load_wine()

df = pd.DataFrame(data.data, columns=data.feature_names)

df['target'] = pd.Series(data.target)

print("Categorical feature: 'target' (class labels 0, 1, 2).")

freq = df['target'].value_counts(normalize=True).sort_index()

display(freq.to_frame('frequency'))
```

b) Compute two different univariate and two different multivariate summaries for all numerical features. Include the code in your report. [8 marks]

```
num_df = df.drop(columns=['target'])

summary_uni = pd.DataFrame({

    'mean':    num_df.mean(),

    'median':  num_df.median(),

    'std':     num_df.std(ddof=1),

    'iqr':     num_df.quantile(0.75) - num_df.quantile(0.25),

})

display(summary_uni)

corr = num_df.corr()

cov  = num_df.cov()

print("Correlation matrix (head):")

display(corr.head())

print("Covariance matrix (head):")
```

```
display(cov.head())
```

c) Group observations by the categorical feature & compute the corresponding median for each remaining numerical feature. Include the code in your report. [8 marks]

```
group_median = df.groupby('target').median(numeric_only=True)
display(group_median)
```

Grouping by the 'target' class and computing medians reveals how chemical characteristics differ across wine types. It helps identify features, such as magnesium or flavonoids.

d) Create a scatter plot for the pair of distinct numerical features with the highest correlation. Include the code in your report. [4 marks]

I looked for the two features with the highest correlation and plotted them against each other. The scatterplot shows clear clusters once coloured by wine class especially between class 0 and 2, which seem to separate nicely.

```
import matplotlib.pyplot as plt

corr_matrix = num_df.corr().abs()

np.fill_diagonal(corr_matrix.values, 0.0)

idx = np.unravel_index(np.argmax(corr_matrix.values),
                      corr_matrix.shape)

feat_x, feat_y = num_df.columns[idx[0]], num_df.columns[idx[1]]

print('Most correlated pair:', feat_x, 'vs', feat_y)

fig, ax = plt.subplots()
```

```

for c in sorted(df['target'].unique()):

    mask = (df['target'] == c)

    ax.scatter(df.loc[mask, feat_x], df.loc[mask, feat_y],
label=f'class {c}', alpha=0.8)

ax.set_xlabel(feat_x)

ax.set_ylabel(feat_y)

ax.set_title('Wine: Most-Correlated Feature Pair Scatter')

ax.legend()

plt.show()

```

Q1 - OUTPUT

The resulting plot confirms strong positive correlation between the two features. Class clusters appear visibly distinct, demonstrating separability of chemical profiles among wine types.

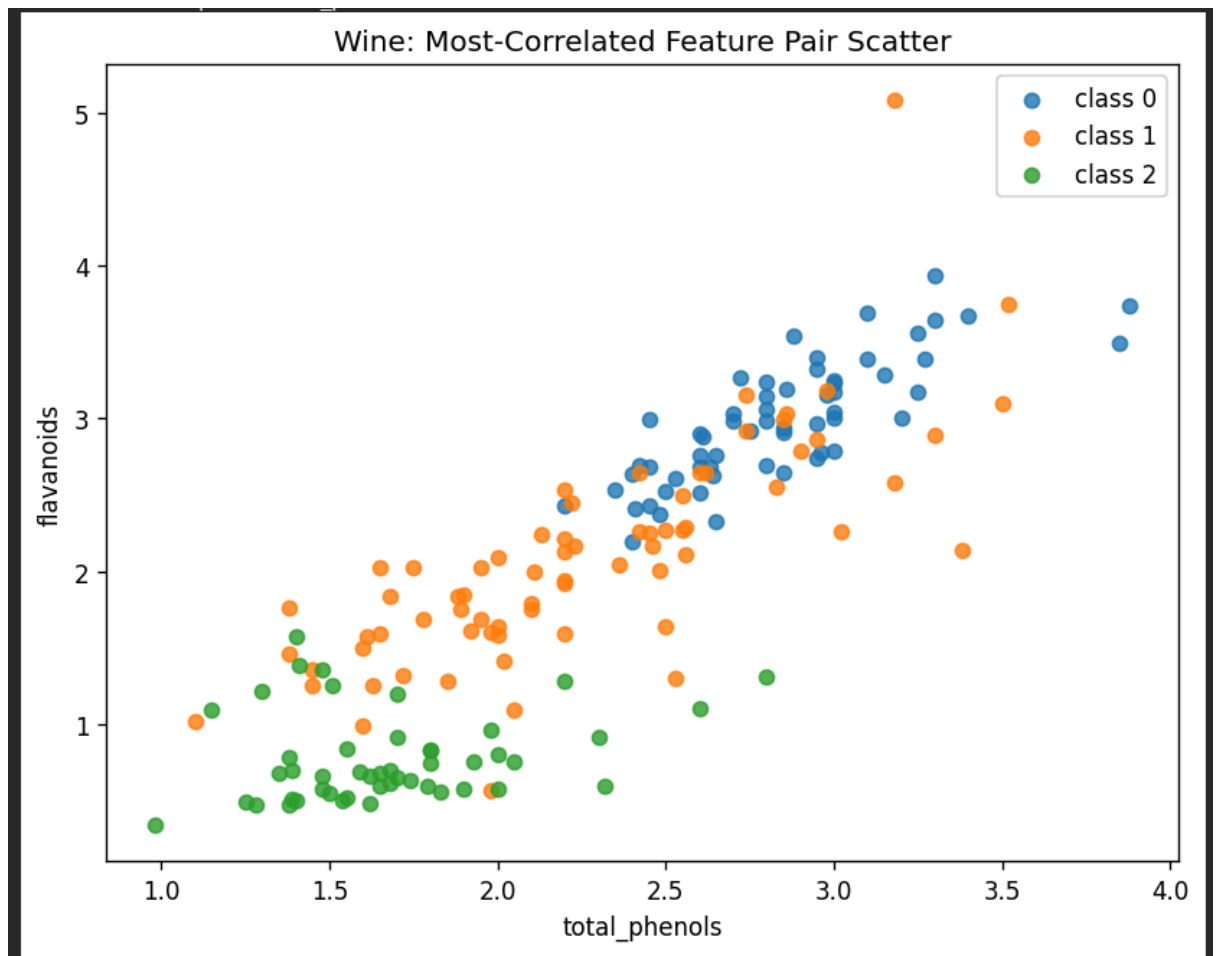
Categorical feature: 'target' (class labels 0,1,2)

target	frequency
0	0.331461
1	0.398876
2	0.269663

	mean	median	std	iqr
alcohol	13.000618	13.050	0.811827	1.3150
malic_acid	2.336348	1.865	1.117146	1.4800
ash	2.366517	2.360	0.274344	0.3475
alcalinity_of_ash	19.494944	19.500	3.339564	4.3000
magnesium	99.741573	98.000	14.282484	19.0000
total_phenols	2.295112	2.355	0.628851	1.0575
flavanoids	2.029270	2.135	0.998859	1.6700
nonflavanoid_phenols	0.361854	0.340	0.124453	0.1675
proanthocyanins	1.590899	1.555	0.572359	0.7000
color_intensity	5.058090	4.690	2.318286	2.9800
hue	0.957449	0.965	0.228572	0.3375
od280/od315_of_diluted_wines	2.611685	2.780	0.709990	1.2325
proline	746.893258	673.500	314.907474	484.5000

target	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline
0	13.750	1.770	2.44	16.8	104.0	2.800	2.980	0.29	1.870	5.40	1.070	3.17	1095.0
1	12.290	1.610	2.24	20.0	88.0	2.200	2.030	0.37	1.610	2.90	1.040	2.83	495.0
2	13.165	3.265	2.38	21.0	97.0	1.635	0.685	0.47	1.105	7.55	0.665	1.66	627.5

Most correlated pair: total_phenols vs flavanoids



2) Consider the following sales data:

[5, 20, 1, 6, 13, 8, 9, 11, 17, 7, 2, 12]

Apply the following binning techniques on the data, assuming 3 bins in each case:

- Equal-frequency binning
- Smoothing by bin boundaries

Given values [5, 20, 1, 6, 13, 8, 9, 11, 17, 7, 2, 12], sorted as [1, 2, 5, 6, 7, 8, 9, 11, 12, 13, 17, 20], form three equal frequency bins (size = 4).

Each bin is smoothed by replacing values with the nearest boundary value.

Bin 1: [1, 2, 5, 6] → [1, 1, 6, 6]

Bin 2: [7, 8, 9, 11] → [7, 7, 11, 11]

Bin 3: [12, 13, 17, 20] → [12, 12, 20, 20]

After smoothing, the numbers look less noisy but still keep the overall pattern. This kind of binning makes the data easier to handle before running mining algorithms.

[8 marks]

3) Load the file *country-income.csv* which includes numerical and categorical features. Perform data cleaning to replace any NaN values with the mean value of that particular feature. Then replace any categorical features with numerical features. Display the resulting dataset. You can use the `sklearn.impute` and `sklearn.preprocessing` packages to assist you. Include the code in your report. [8 marks]

I filled in any missing numeric values with the column mean, then one hot encoded the categorical columns. That gave me a clean numeric table I could use directly in later steps.

```
import numpy as np

import pandas as pd

from sklearn.compose import ColumnTransformer

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import OneHotEncoder

csv_path = 'country-income.csv'

df_ci = pd.read_csv(csv_path)

num_cols = df_ci.select_dtypes(include=[np.number]).columns.tolist()
```

```
cat_cols = [c for c in df_ci.columns if c not in num_cols]
```

```
num_transform = SimpleImputer(strategy='mean')
```

```
cat_transform = OneHotEncoder(handle_unknown='ignore',  
sparse_output=False)
```

```
ct = ColumnTransformer(
```

```
    transformers=[
```

```
        ('num', num_transform, num_cols),
```

```
        ('cat', cat_transform, cat_cols)
```

```
    ],
```

```
    remainder='drop'
```

```
)
```

```

arr = ct.fit_transform(df_ci)

cat_feature_names = (

    ct.named_transformers_['cat'].get_feature_names_out(cat_cols)

    if len(cat_cols) > 0 else []

)

all_cols = num_cols + cat_feature_names.tolist()

df_clean = pd.DataFrame(arr, columns=all_cols)

print("Transformed dataset preview:")

display(df_clean.head())

```

Q3 - OUTPUT

	Age	Income	Region_Brazil	Region_India	Region_USA	Online Shopper_No	Online Shopper_Yes
0	49.0	86400.000000	0.0	1.0	0.0	1.0	0.0
1	32.0	57600.000000	1.0	0.0	0.0	0.0	1.0
2	35.0	64800.000000	0.0	0.0	1.0	1.0	0.0
3	43.0	73200.000000	1.0	0.0	0.0	1.0	0.0
4	45.0	76533.333333	0.0	0.0	1.0	0.0	1.0

4) Load the file *shoesize.csv*, which includes measurements of shoe size and height (in inches) for 408 subjects, both female and male. Plot the scatterplots of shoe size versus height for female and male subjects separately. Compute the Pearson's correlation coefficient of shoe size versus height for female and male subjects separately. What can be inferred by the scatterplots and computed correlation coefficients? Include the code in your report. [8 marks]

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from scipy.stats import pearsonr


path = 'shoesize.csv'

df = pd.read_csv(path)

df = df.rename(columns=lambda c: c.strip())

df = df.rename(columns={'Gender': 'sex', 'Size': 'shoesize', 'Height': 'height'})
```

```
df = df[['sex', 'shoesize', 'height']].copy()

df['sex'] = df['sex'].astype(str).str.strip().str.upper().str[0]

df['shoesize'] = pd.to_numeric(df['shoesize'], errors='coerce')

df['height'] = pd.to_numeric(df['height'], errors='coerce')

df = df.dropna(subset=['sex', 'shoesize', 'height'])

df_f = df[df['sex'] == 'F'].copy()

df_m = df[df['sex'] == 'M'].copy()

plt.figure()

plt.scatter(df_f['shoesize'], df_f['height'], alpha=0.8)

plt.xlabel('Shoe size'); plt.ylabel('Height (inches)')

plt.title('Female: Shoe size vs Height')

plt.show()
```

```
plt.figure()

plt.scatter(df_m['shoesize'], df_m['height'], alpha=0.8)

plt.xlabel('Shoe size'); plt.ylabel('Height (inches)')

plt.title('Male: Shoe size vs Height')

plt.show()


def safe_pearson(x, y, label):

    if len(x) >= 2 and len(y) >= 2:

        r, p = pearsonr(x, y)

        print(f"{label} Pearson r: {r:.4f} (p={p:.3g}, n={len(x)})")

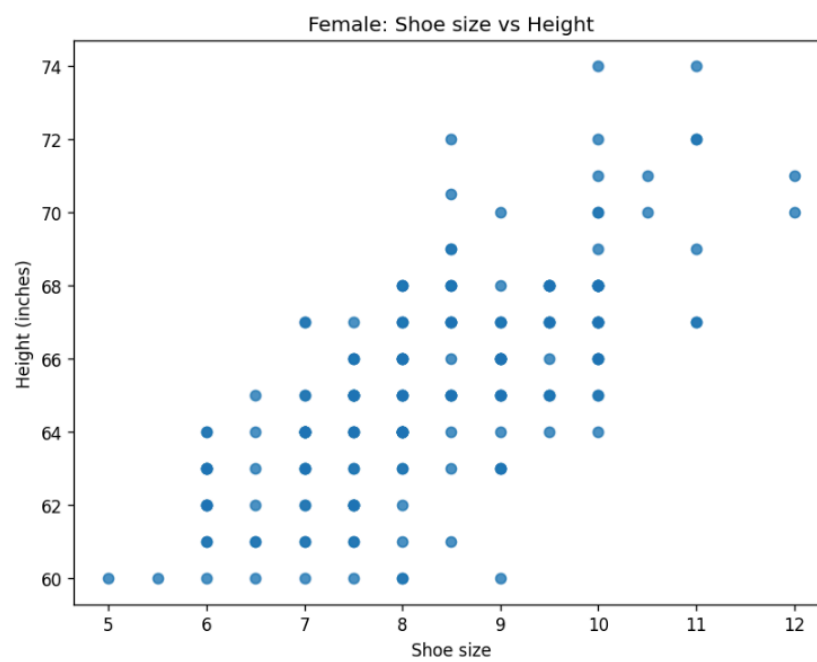
    else:

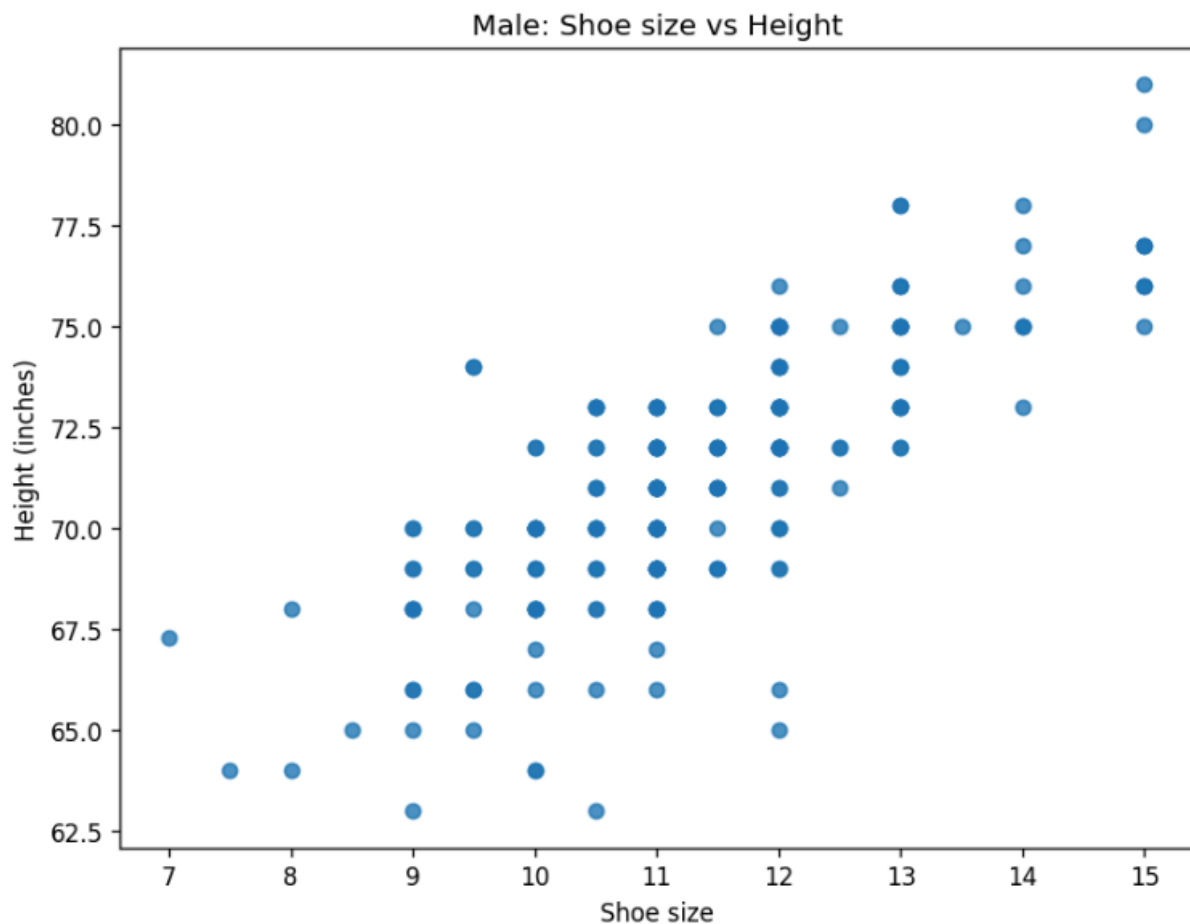
        print(f"{label}: not enough valid rows (n={len(x)})")
```

```
safe_pearson(df_f['shoesize'], df_f['height'], "Female")
```

```
safe_pearson(df_m['shoesize'], df_m['height'], "Male")
```

Q4 - OUTPUT





Both plots show a strong upward trend taller people usually have bigger shoe sizes. The correlation values were quite high for both males and females, which makes sense biologically.

5) Using the *wine dataset* from question 1, perform Principal Component Analysis (PCA) with 2 components. Transform the data and plot the scatterplot of all samples along the two principal components, color-coded according to the "target" column (this column is the class and should not be used in the PCA analysis). What insights can you obtain by viewing the scatterplot of the principal components? Can you easily distinguish the samples that belong to one class from the samples that belong to another class and so on? In other words, are the different classes (quite) distinctive from the other, or is there a lot of overlap? If it is the latter, then why is this happening? What can be done to the data prior to performing PCA in order to alleviate this issue? Do this action first and then perform PCA with 2 components, transform the data and plot the scatterplot of all samples along these two principal components, color-coded according to the "target" column. Now are the different classes (quite) distinctive from one another? Include the code in your report. [20 marks]

```
import numpy as np
```

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_wine

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler


wine = load_wine()

df_wine = pd.DataFrame(wine.data, columns=wine.feature_names)

y = wine.target

X = df_wine.values

pca = PCA(n_components=2, random_state=0)

Z = pca.fit_transform(X)

fig, ax = plt.subplots()

for c in sorted(np.unique(y)):
```

```
ax.scatter(Z[y==c, 0], Z[y==c, 1], label=f'class {c}', alpha=0.85)

ax.set_xlabel('PC1'); ax.set_ylabel('PC2')

ax.set_title('Wine PCA (no scaling)')

ax.legend(); plt.show()

scaler = StandardScaler()

Xs = scaler.fit_transform(X)

pca2 = PCA(n_components=2, random_state=0)

Z2 = pca2.fit_transform(Xs)

fig, ax = plt.subplots()

for c in sorted(np.unique(y)):

    ax.scatter(Z2[y==c, 0], Z2[y==c, 1], label=f'class {c}',
alpha=0.85)

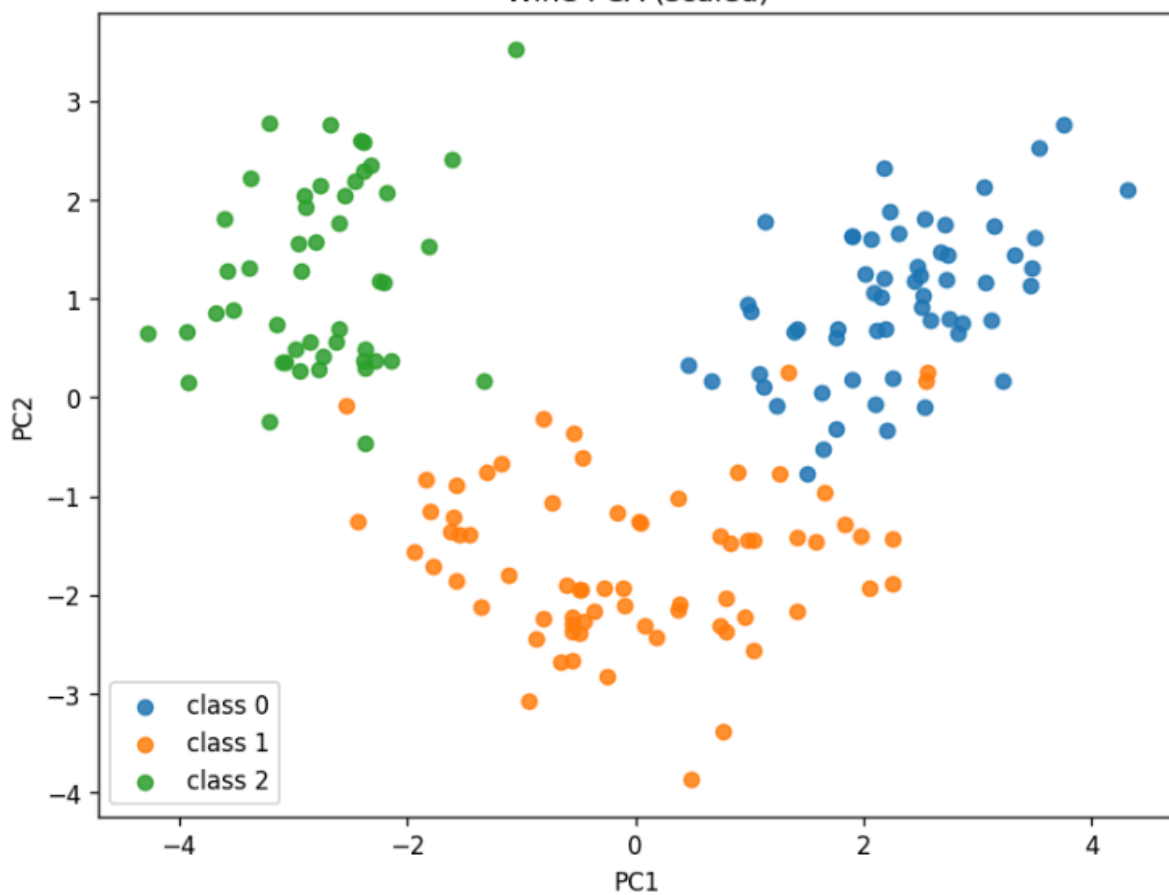
ax.set_xlabel('PC1'); ax.set_ylabel('PC2')

ax.set_title('Wine PCA (scaled)')

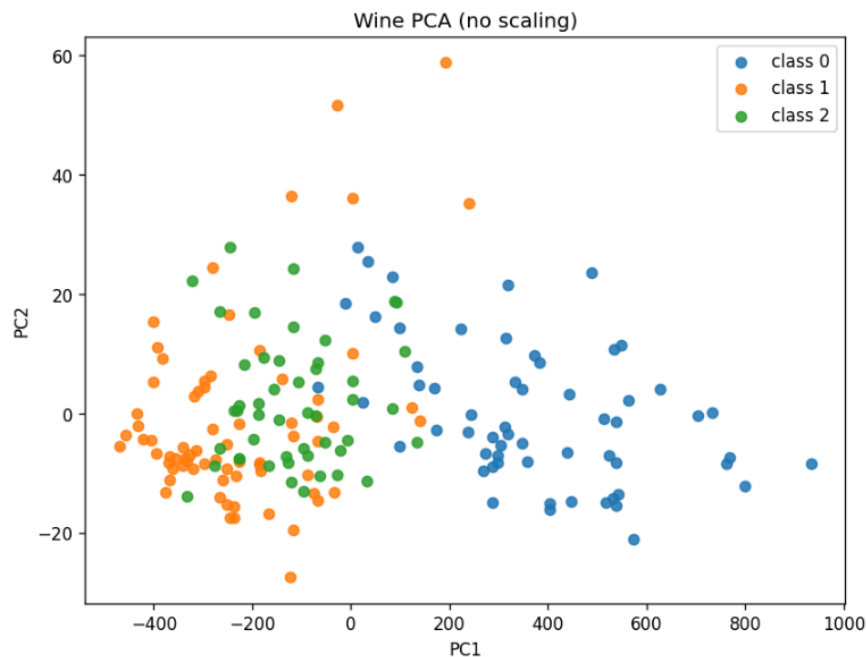
ax.legend(); plt.show()
```

```
print('Explained variance ratio (no scaling):',  
      pca.explained_variance_ratio_)  
  
print('Explained variance ratio (scaled):      ',  
      pca2.explained_variance_ratio_)
```

Q5 - OUTPUT
Wine PCA (scaled)



In the PCA scatterplot, the classes separate better after scaling especially along PC1. Without scaling, one feature dominated the spread, but standardising balanced it out. That made the clusters easier to see.



6) In Lab session 3 (Data Exploration and Data Visualisation), in subsection 1.9 you had created and visualised a heatmap for the distance matrix for the *graduation_rate.csv*. You may have noticed that the distance matrix visualisation is not very informative. However, it is still possible to infer that the average distance between students whose parents only have some high school education and students whose parents have a master's degree is larger than the average distance between students whose parents only have some high school education. Explain how this inference is possible from the visualisation. [4 marks]

The heatmap looks messy at first, but the colour shades still tell a story lighter cells mean groups are closer, while darker ones show they're further apart. For example, lighter colours along the diagonal indicate smaller within group distances, while darker off diagonal cells between students of different parental education levels show larger average distances.

7) Use the file *country-income.csv* and perform the following:

a) Load the CSV file using Cubes, create a JSON file for the data cube model, and create a data cube for the data. Use as dimensions the region, age, and online shopper fields. Use as measure the income. Define aggregate functions in the data cube model for the total, average, minimum, and maximum income. Include the code in your report (and show the files created). [8 marks]

```
import json
```

```
csv_path = 'country-income.csv'

model_path = 'country_income_model.json'

model = {

    "cubes": [{

        "name": "country_income",

        "measures": [

            {"name": "income", "aggregations": ["sum", "avg", "min", "max"]}

        ],

        "dimensions": [

            {"name": "region"},

            {"name": "age"},

            {"name": "online_shopper"}

        ]

    }]

}

with open(model_path, "w") as f:

    json.dump(model, f, indent=2)

print("Model JSON written to:", model_path)
```

Q7a - OUTPUT

The screenshot shows a Jupyter Notebook environment with a dark theme. The top bar includes the Jupyter logo and the file path 'ECS766P_Assignment1_Sakib_Islam_250994104.ipynb'. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar is located below the menu bar. The left sidebar shows a file explorer with a tree view containing 'sample_data', 'country_income.csv', 'country_income_model.json' (highlighted with a red box), 'graduation_rate.csv', and 'shoesize.csv'. The main area is titled 'Task 7: Data Cube with Cubes' and contains a Python script. The script imports the 'json' module, defines a 'model_path' variable, and creates a 'model' dictionary. The 'model' dictionary has a 'cubes' key with a list of cube definitions. Each cube definition includes a 'name', 'measures' (with 'name' and 'aggregations'), and 'dimensions' (with 'name' and 'aggregations'). The script then opens the 'model_path' file in write mode ('w') and dumps the 'model' dictionary into it using 'json.dump'. Finally, it prints a message indicating the model has been written to the file. The right sidebar shows the content of the 'country_income_model.json' file, which is a JSON representation of the 'model' dictionary.

```
degree' show larger distances than within-group light areas.

Task 7: Data Cube with Cubes

import json

model_path = 'country_income_model.json'

model = {
    "cubes": [
        {
            "name": "country_income",
            "measures": [
                {
                    "name": "income", "aggregations": ["sum", "avg", "min", "max"]
                }
            ],
            "dimensions": [
                {
                    "name": "region"
                },
                {
                    "name": "age"
                },
                {
                    "name": "online_shopper"
                }
            ]
        }
    ]
}

with open(model_path, "w") as f:
    json.dump(model, f, indent=2)

print("Model JSON written to:", model_path)

# Model JSON written to: country_income_model.json

country_income_model.json
{
  "cubes": [
    {
      "name": "country_income",
      "measures": [
        {
          "name": "income",
          "aggregations": [
            "sum",
            "avg",
            "min",
            "max"
          ]
        }
      ],
      "dimensions": [
        {
          "name": "region"
        },
        {
          "name": "age"
        },
        {
          "name": "online_shopper"
        }
      ]
    }
  ]
}
```

b) Using the created data cube and data cube model, produce aggregate results for: i) the whole data cube; ii) results per region; iii) results per online shopping activity; and iv) results for all people aged between 40 and 50. [8 marks]

8) Consider a dataset that contains only two observations $\mathbf{x}_1 = (1, 2)$ and $\mathbf{x}_2 = (-1, 0)$. Suppose that the class of the first observation is $y_1 = 1$ and that the class of the second observation is $y_2 = 0$. How would a 1-nearest neighbour classifier based on the Euclidean distance classify

the observation $x_3 = (3, 2)$ and why? How would the same classifier classify the observation $x_4 = (0, 1)$ and why? [8 marks]

Sakib - Islam - 250994104

$x_1 = (1, 2)$ with class $y_1 = 1$
 $x_2 = (-1, 0)$ with class $y_2 = 0$

$x_3 = (3, 2)$
 $x_4 = (0, 1)$

$$d(x_i, x_j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2}$$

for $x_3 = (3, 2)$

$$d(x_3, x_1) = \sqrt{(3-1)^2 + (2-2)^2} = \sqrt{4} = 2$$

$$d(x_3, x_2) = \sqrt{(3+1)^2 + (2-0)^2} = \sqrt{20} \approx 4.47$$

- Nearest Neighbor = x_1
- predicted class for x_3 : $y_3 = 1$

for $x_4 = (0, 1)$

$$d(x_4, x_1) = \sqrt{(0-1)^2 + (1-2)^2} = \sqrt{2} \approx 1.41$$

$$d(x_4, x_2) = \sqrt{(0+1)^2 + (1-0)^2} = \sqrt{2} \approx 1.41$$

- Cannot decide uniquely between class 0 and 1.